I. THE LIMITATIONS OF RELATIONAL DATABASES (RDBMS)

Traditional databases like MySQL are "ACID" compliant, ensuring 100% data integrity. However, when handling **Petabytes** of data or **millions of requests per second**, RDBMS face two major bottlenecks:

1. **Vertical Scaling Only:** To handle more data, you need a bigger, more expensive server. You cannot easily spread one SQL table across 100 cheap servers.
2. **Rigid Schema:** In Big Data, data formats change constantly. SQL requires a fixed schema, making it difficult to store diverse data types like JSON, images, or sensor pings.

NoSQL (Not Only SQL) was designed to solve these issues by prioritizing **Horizontal Scalability** (adding more cheap servers to a cluster) and **Schema Flexibility**.

II. THE CAP THEOREM: THE GOLDEN RULE OF DISTRIBUTED SYSTEMS

In a distributed Big Data environment, a database can only provide **two out of three** guarantees:

- **C (Consistency):** Every read receives the most recent write or an error.
- **A (Availability):** Every request receives a (non-error) response, without the guarantee that it contains the most recent write.
- **P (Partition Tolerance):** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

Note: In Big Data, we **must** choose **P**. Therefore, NoSQL databases are either **CP** (like MongoDB) or **AP** (like Cassandra).

III. MONGODB: THE DOCUMENT-ORIENTED LEADER (CP)

MongoDB stores data in **BSON** (Binary JSON) format. It is the most popular NoSQL choice for developers because its "Document" model maps directly to objects in programming languages like Python.

3.1. Key Features

- **Schema-less:** One document in a collection can have 5 fields, while the next has 50.
- **Dynamic Indexing:** Fast retrieval of nested data within a JSON structure.
- **Horizontal Scaling (Sharding):** Splitting a massive collection across multiple servers based on a "Shard Key."

3.2. Use Cases

- Content Management Systems (CMS).
- Real-time Analytics dashboards.
- Storing User Profiles with varying attributes.

IV. APACHE CASSANDRA: THE WIDE-COLUMN TITAN (AP)

Originally developed at Facebook, Cassandra is designed for **Zero Downtime** and **Massive Write Throughput**. It uses a "Masterless" architecture where every node in the cluster is equal.

4.1. Key Features

- **High Availability:** If 3 out of 10 servers crash, the system still functions perfectly.
- **Linear Scalability:** Doubling the number of nodes doubles the performance.
- **CQL (Cassandra Query Language):** Looks like SQL but is designed for distributed lookups.

4.2. Use Cases

- IoT Sensor Data (Writing billions of pings per hour).
- Messaging apps (WhatsApp/Discord style history).
- Fraud detection logs in banking.

V. COMPARISON TABLE: SQL VS. MONGODB VS. CASSANDRA

Feature	Relational (SQL)	MongoDB (Document)	Cassandra (Wide-Column)
Data Model	Tables/Rows	JSON-like Documents	Rows/Columns (Wide)
Scaling	Vertical (Bigger Server)	Horizontal (Sharding)	Horizontal (Masterless)
Primary Goal	Data Integrity (ACID)	Development Speed	High Availability
Query Lang.	SQL	MQL (JSON-based)	CQL
Best For	Financial Transactions	Web Apps / RAG Data	High-speed Logging/IoT

VI. PRACTICAL LAB: MONGODB FOR RAG SYSTEMS

In a RAG system, MongoDB is often used to store the **Metadata** of your documents (Author, Date, Category) alongside the text content.

Python

```
from pymongo import MongoClient


# 1. Connect to Cluster
client = MongoClient("mongodb://localhost:27017/")
db = client.dtu_ai_library
```

```
collection = db.research_papers

# 2. Insert a Document (Schema-less)
paper = {
    "title": "Attention is All You Need",
    "year": 2017,
    "tags": ["Transformers", "NLP", "Deep Learning"],
    "authors": [{"name": "Ashish Vaswani"}]
}
collection.insert_one(paper)

# 3. Query based on nested tags
results = collection.find({"tags": "NLP"})
for doc in results:
    print(doc["title"])
```